



اُونِيْوَرْسِيْتِيْ تِيْكْنُوْلُوْجِيْ مَآرَا
UNIVERSITI
TEKNOLOGI
MARA

FACULTY :

FACULTY OF SCIENCE COMPUTER AND MATHEMATIC

SUBJECT PROGRAM CODE :

ITT626 - BACK-END TECHNOLOGY

CLASS | GROUP NAME | PROJECT NAME :

M3CDCS2515A

CASE STUDY

LECTURER NAME :

IDAYATI MAZLAN

MOHAMAD HAZIQ NAQIB BIN ZAID	2025181477
------------------------------	------------

PREPARED FOR : IDAYATI MAZLAN

1. PROJECT BACKGROUND	3
2. PROJECT MODULES	4
3. USE CASE OF THE PROJECT	6
4. FLOW CHART OF THE PROJECT	8
5. ACTIVITY DIAGRAM OF THE PROJECT	11
6. ENTITY RELATIONSHIP DIAGRAM (ERD)	13
7. USER INTERFACE	15

1. PROJECT BACKGROUND

The project is centered on the development of an automated, back-end-driven ecosystem designed to modernize the traditional, manual lost and found processes common in high-density environments like the UiTM campus. Currently, students and security personnel rely on fragmented methods such as manual paper logbooks, physical notice boards, and passive social media blasts, all of which are prone to data entry errors and low recovery rates. This proposed solution utilizes a robust two-tier architecture hosted on a VPS, where a Laravel-based back-end serves as the central intelligence hub. The system's primary innovation is its Multi-API Integration, which automates the most tedious aspects of item recovery. By leveraging an existing Telegram Bot as a primary interactive frontend, the project allows users to submit reports via a frictionless chat interface, utilizing the Telegram Bot API for real-time engagement and automated "Match Alerts".

Technically, the back-end manages the "Complexity" of the project through sophisticated data processing and external API calls. When a "Found" report is submitted with a photo, the system triggers an AI Vision Integration (via OpenCV or a Cloud Vision API) to automatically extract descriptive tags such as color and object type. Simultaneously, the Google Maps API is employed to capture and store GPS (Location Based) data, allowing the system to run precise spatial calculations between where an item was lost and where it was recovered. The core of the system is an automated matching engine that performs a rolling calculation to compare these AI-generated tags and coordinates, triggering a notification only when a high-confidence match (over 80%) is detected. This ensures that users receive hyper-targeted alerts rather than being overwhelmed by irrelevant data.

The scope of the project encompasses several distinct modules to ensure a complete system flow that meets all end-user requirements. For students, the Finder and Loser modules provide simple tools for reporting and subscription through the bot. For administrative staff, a web-based Management Dashboard allows Campus Security to oversee the physical inventory, retrieve images from the database for visual verification, and manage a secure handover process. This handover is further secured by the bot, which can generate a One-Time Password (OTP) for the claimant to verify their identity at the security desk. By integrating these advanced back-end technologies, the project aims to replace inefficient manual labor with a high-accuracy, automated recovery system that can be monitored through comprehensive API logs and managed via a structured ERD-based database design.

2. PROJECT MODULES

The system is divided into several modules to support the functionalities required by customers and administrators/mechanics.

2.1 Finder Module (Telegram Bot Interface)

This module is optimized for frictionless data entry for students who find lost items on campus.

- **Report Submission:** Users initiate a report by sending the /found command to the Telegram Bot.
- **Image Upload:** The bot prompts the user to take a photo of the item, which is then sent via the Telegram Bot API and stored as an image in the database.
- **Location Sharing:** The user shares their current GPS (Location Based) data through the bot, which the back-end processes via the Google Maps API.
- **Automated Tagging:** Once the image is received, the back-end triggers the OpenCV/Vision API to automatically extract descriptive tags (e.g., "wallet," "brown").

2.2 Loser Module (Web & Telegram Bot Interface)

This module allows students who have lost items to create search profiles and subscribe to automated alerts.

- **Lost Item Registration:** Users can fill out a form on the web dashboard or via the bot to describe their missing item.
- **Spatial Pinning:** Using the Google Maps API on the web dashboard, users can drop a pin on a map to indicate the last known location of the item.
- **Subscription Management:** Users can subscribe to specific keyword alerts (e.g., "Black iPhone") through the Telegram bot.
- **Instant Match Alerts:** When a match is found, the system performs a calculation and instantly sends a push notification to the user's Telegram.

2.3 Campus Security Module (Web Dashboard)

This module provides administrative control for staff to manage physical inventory and verify owners.

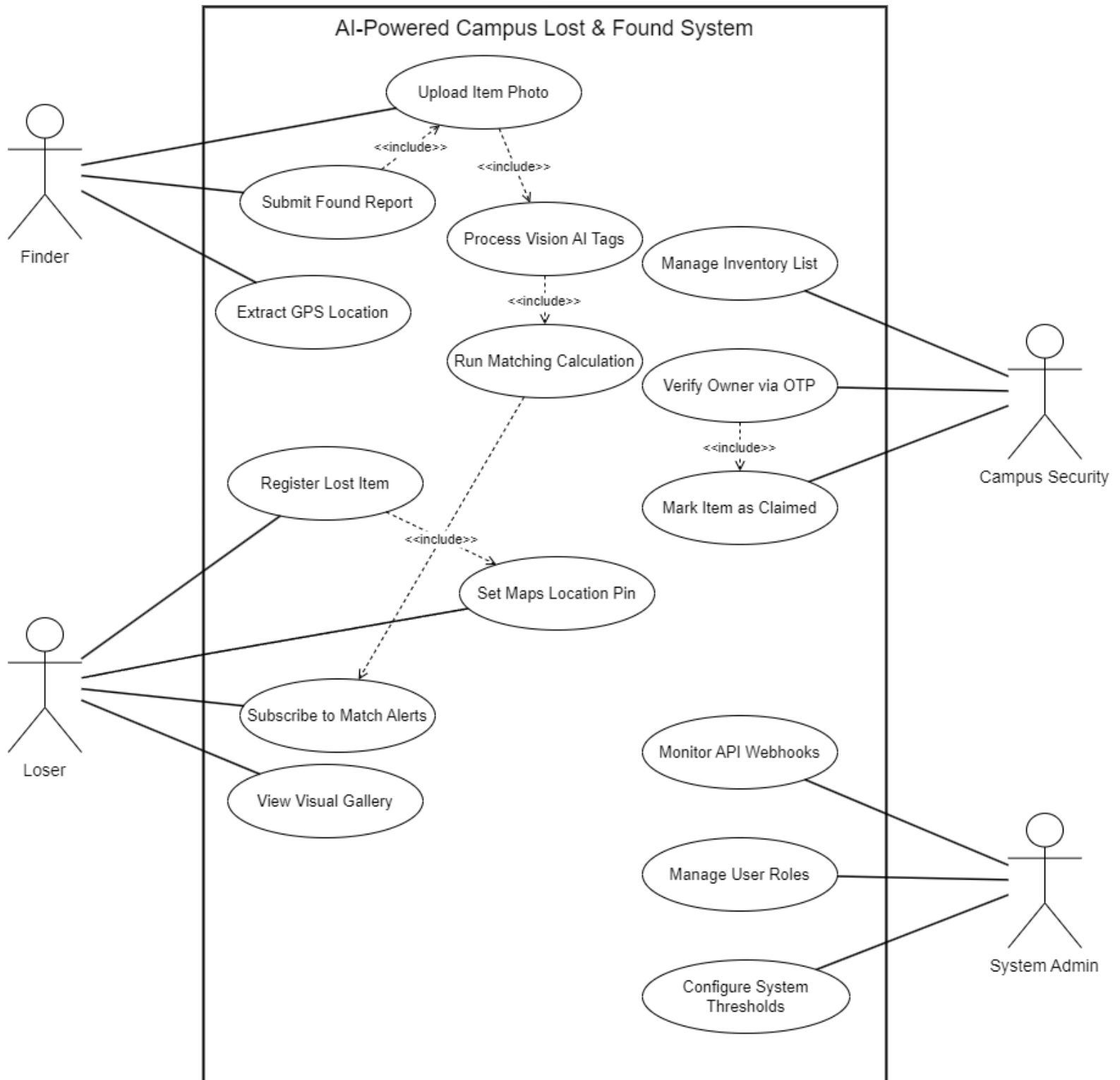
- **Inventory Management:** Security staff can view a comprehensive table of all "Found" items currently held at their post.
- **Visual Verification:** Staff can retrieve images from the database to compare a user's description with the actual item found.
- **Secure Handover (OTP):** When a user claims an item, staff triggers an OTP through the Telegram Bot API to the claimant's phone to verify their identity.
- **Claim Tracking:** Full CRUD capability to update the status of an item from "Pending" to "Claimed" or "Returned".

2.4 System Administrator Module (Web Dashboard)

This module focuses on the back-end technology maintenance and system health.

- **API Monitoring:** Access to API logs to monitor the performance and usage of Google Maps, OpenCV, and Telegram webhooks.
- **User Management:** The ability to verify user profiles and manage permissions for different security posts across campus.
- **Database Oversight:** Directly manage the Entity Relationship Diagram (ERD) data to ensure all records are synchronized and accurate.
- **Matching Engine Control:** Ability to adjust the "confidence score" threshold (e.g., changing it from 80% to 75%) for the automated matching calculation.

3. USE CASE OF THE PROJECT



This Use Case Diagram illustrates the functional requirements and actor interactions within the AI-Powered Campus Lost & Found System. It defines the system boundary and maps out how four primary actors Finder, Loser, Campus Security, and System Admin engage with the platform's features.

1. Student Interactions (Finder & Loser)

- The Finder: Interacts with the system to Submit a Found Report. As shown by the direct association lines, this role is also responsible for explicitly interacting with the Upload Item Photo and Extract GPS Location sub-tasks.
- The Loser: Engages with the system to Register a Lost Item, which involves manually Setting a Maps Location Pin. They also manage their notifications by interacting with Subscribe to Match Alerts and browsing the View Visual Gallery.

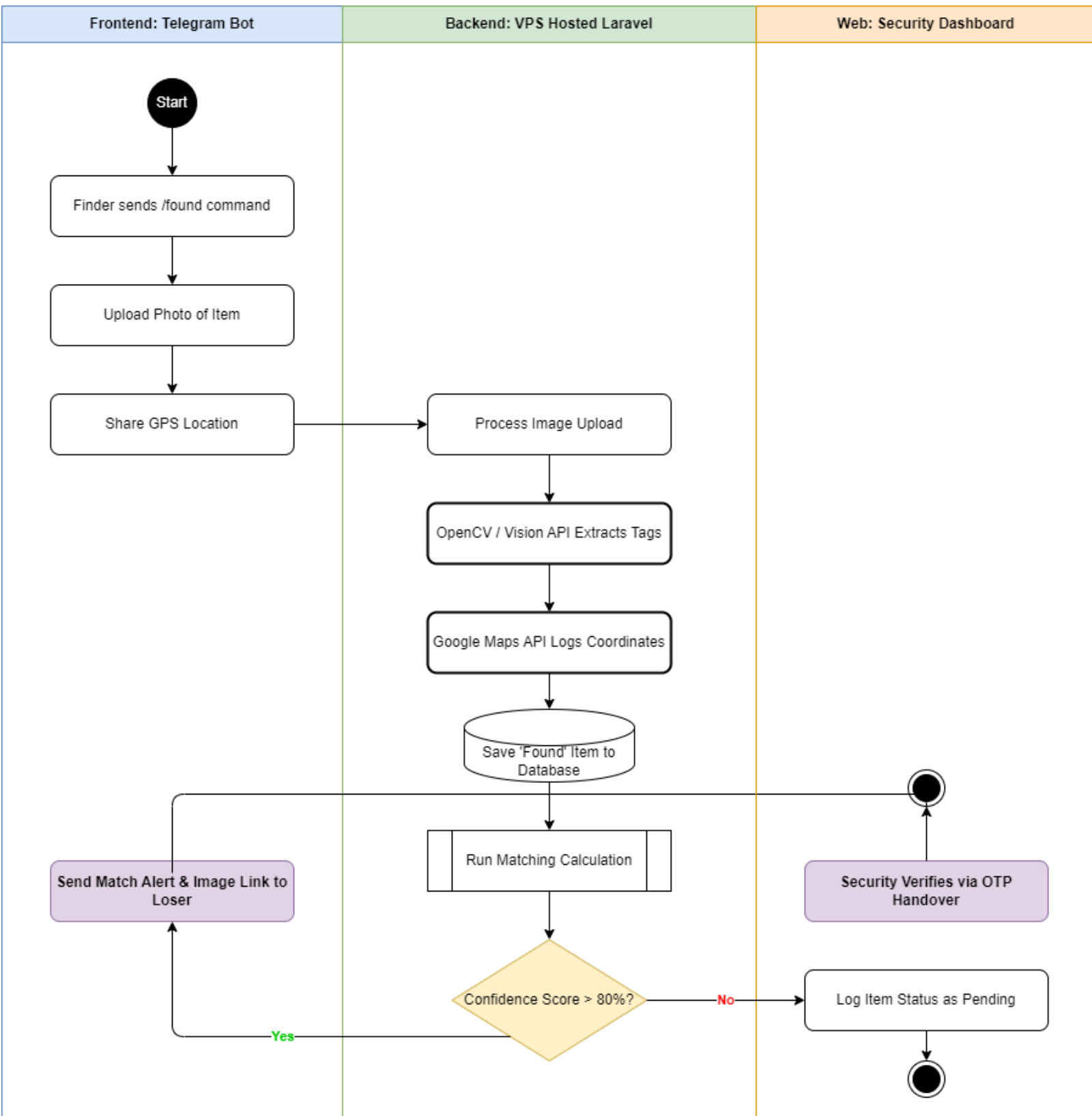
2. Automated System Workflow (<<include>> & <<trigger>>) The diagram highlights the back-end complexity through system dependencies:

- When a photo is uploaded, it automatically forces (<<include>>) the Process Vision AI Tags function.
- Once tags are generated, the system mandatorily executes (<<include>>) the Run Matching Calculation.
- If the calculation yields a high confidence score, the system automatically <<trigger>>s the Subscribe to Match Alerts use case, pushing a notification to the Loser without any human intervention.

3. Administrative & Security Operations

- Campus Security: Acts as the operational authority. They have direct access to Manage the Inventory List and Verify Owner via OTP. The verification process logically <<include>>s the final step of Marking the Item as Claimed.
- System Admin: Focuses purely on back-end health and configuration. They are connected to technical use cases such as Monitor API Webhooks, Manage User Roles, and Configure System Thresholds (e.g., adjusting the AI matching confidence score).

4. FLOW CHART OF THE PROJECT



The activity diagram utilizes a cross-functional flowchart (swimlane) structure to illustrate the end-to-end operational workflow of the AI-Powered Campus Lost & Found System. By dividing the process into three distinct tiers which is the Telegram Bot, the VPS-Hosted Laravel Back-End, and the Security Web Dashboard, the diagram demonstrates a modern, decoupled architecture.

The workflow is broken down into three primary phases:

1. Phase 1: Front-End Data Capture (Telegram Bot)

The sequence initiates when a Finder interacts with the Telegram bot using the `/found` command. The system guides the user through a frictionless data entry process, prompting them to upload a photo of the item and share their immediate GPS location. This ensures standardized data collection before transmitting the payload to the server.

2. Phase 2: Multi-API Processing (Laravel Back-End)

Once the payload reaches the VPS-hosted Laravel back-end, the system executes its core automated logic without human intervention.

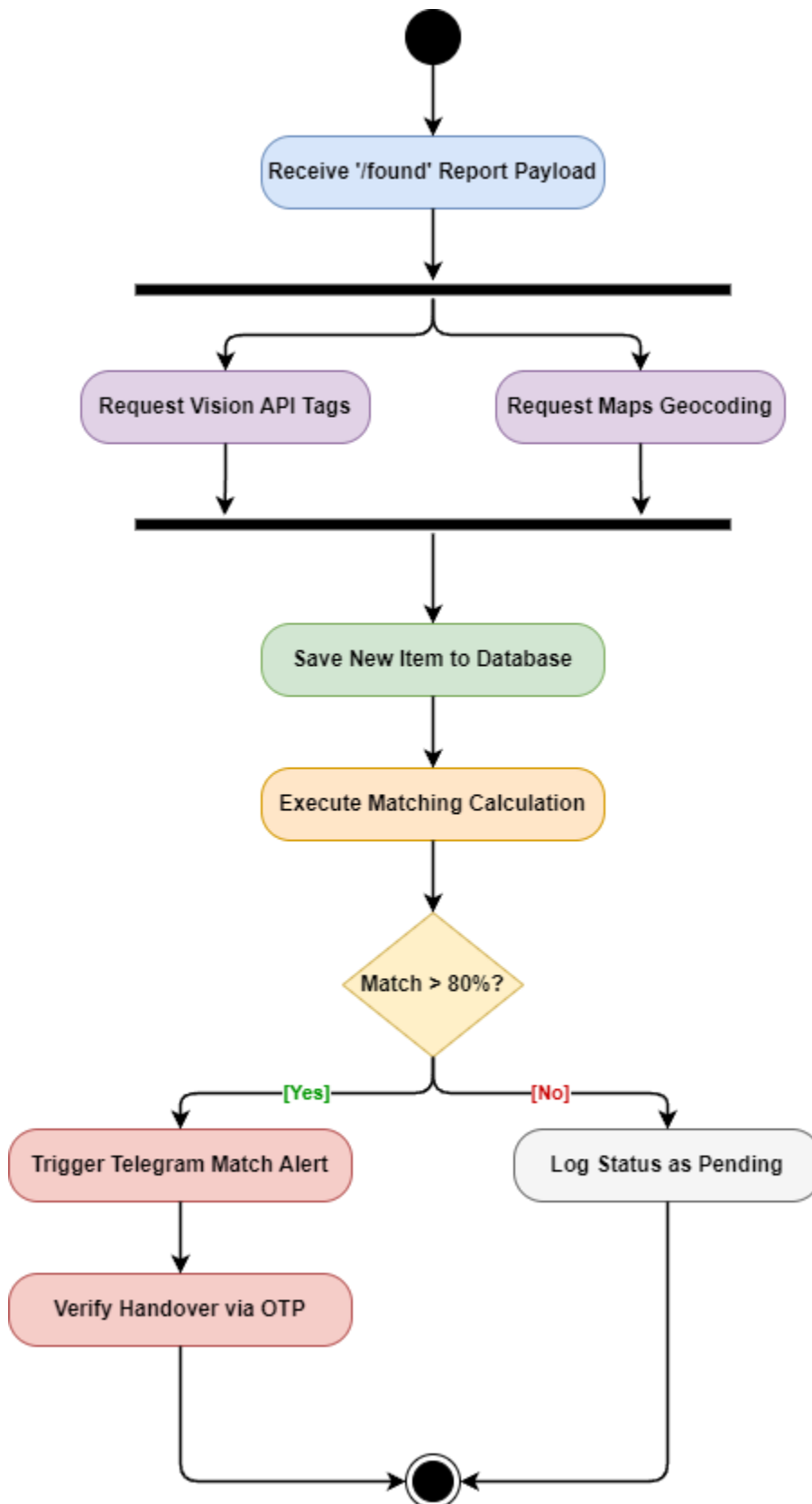
- **API Integration:** The back-end simultaneously routes the uploaded image to the OpenCV / Vision API to extract descriptive automated tags, and sends the GPS coordinates to the Google Maps API for precise spatial logging.
- **Database Storage:** The processed data, along with the generated AI tags, is securely saved as a new "Found" item record in the database.
- **Matching Calculation:** The system's matching engine triggers a mathematical calculation, comparing the new item's tags and location against the active database of "Lost" item reports.

3. Phase 3: Automated Decision & Resolution

The workflow reaches a critical decision node evaluating if the matching calculation yields a confidence score greater than 80%.

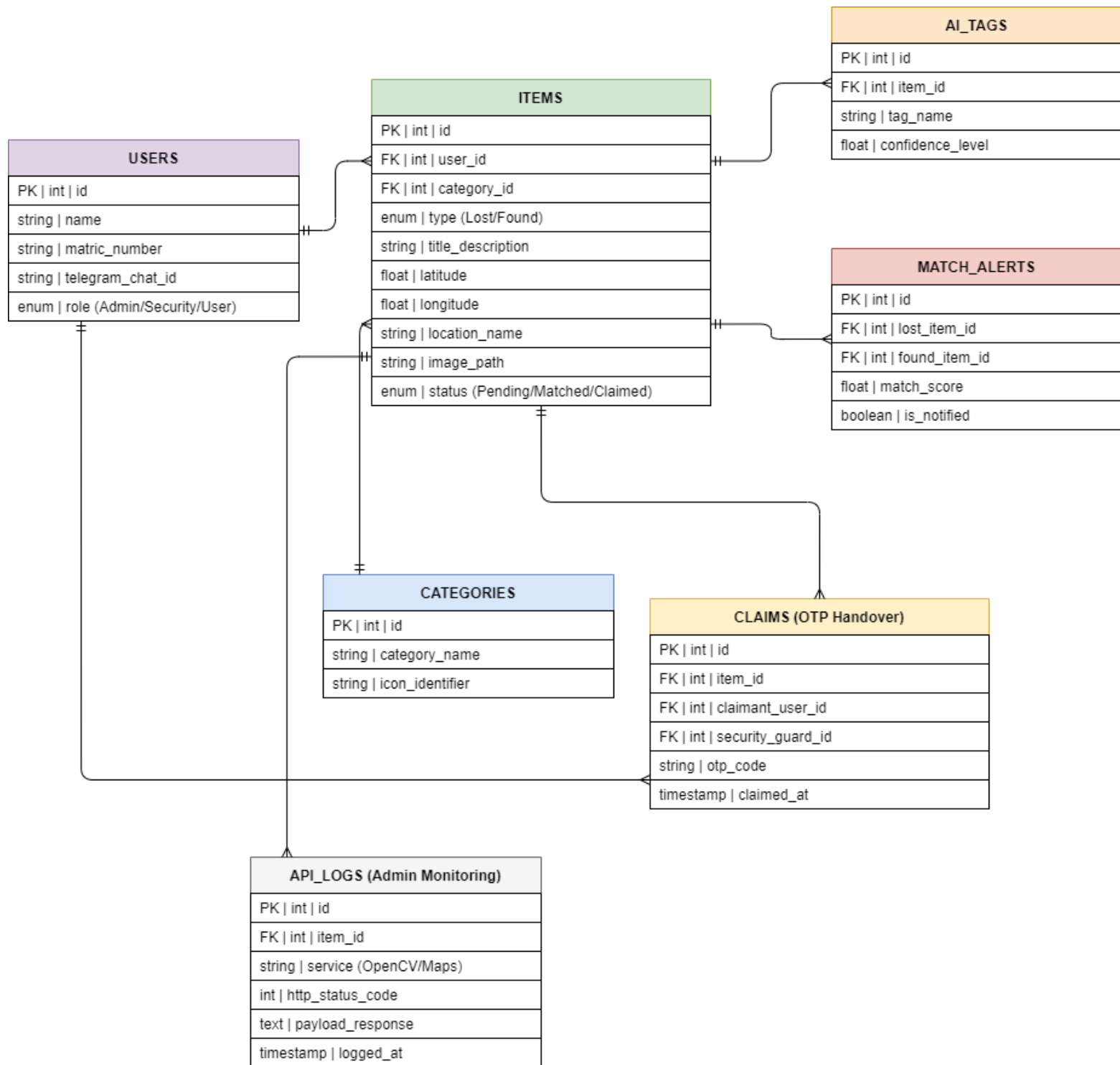
- Negative Outcome (No Match): If the score is below the threshold, the system automatically routes the item to the Security Dashboard lane, logging its status as "Pending" in the inventory for future cross-referencing.
- Positive Outcome (Match > 80%): If a high-confidence match is detected, the back-end instantly triggers a webhook back to the Telegram Bot, sending a direct Match Alert and image link to the original Loser. The workflow then transitions to the Security Dashboard, where Campus Security completes the physical handover by verifying the system-generated OTP provided by the claimant.

5. ACTIVITY DIAGRAM OF THE PROJECT



Unlike a standard flowchart, this UML Activity Diagram utilizes **Fork and Join nodes** (represented by the thick black synchronization bars) to illustrate the system's asynchronous capabilities. When a report payload is received, the back-end does not process tasks sequentially. Instead, it "forks" the process to request tags from the **Vision API** and reverse-geocoding data from the **Google Maps API** simultaneously. Once both APIs respond, the process "joins" back together to save the consolidated item to the database. This parallel processing architecture significantly reduces server latency before executing the final matching calculation and routing the control flow based on the 80% confidence threshold.

6. ENTITY RELATIONSHIP DIAGRAM (ERD)

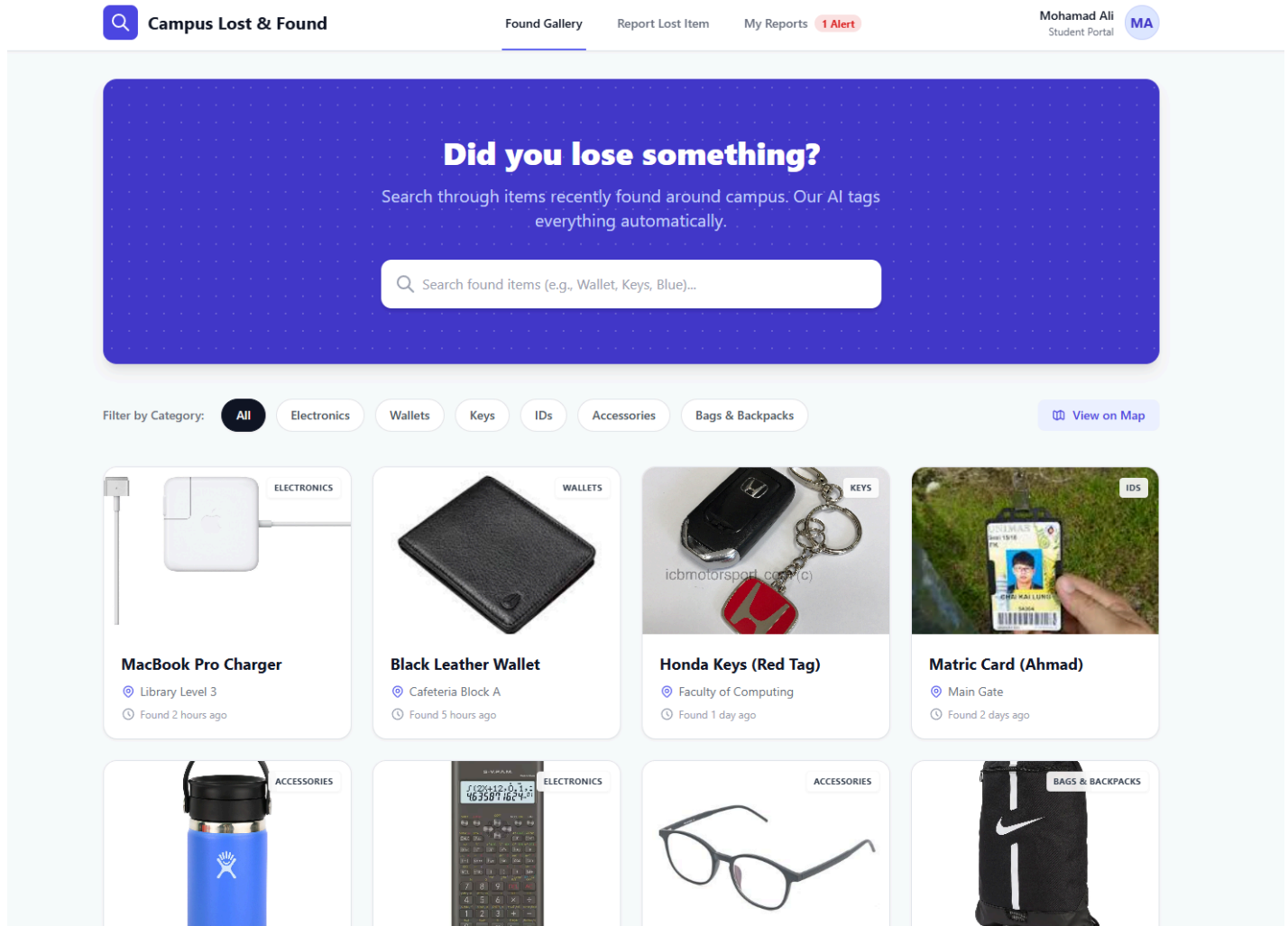


The Entity Relationship Diagram (ERD) illustrates the normalized database schema designed for the AI-Powered Campus Lost & Found System. The architecture is optimized for relational integrity and is structured to fully support the complex, multi-API back-end logic.

The database is built around a central hub with six interconnected relational tables:

- Core Entities (**USERS**, **CATEGORIES**, and **ITEMS**): The **ITEMS** table acts as the central repository, storing essential data such as the VPS image path and Google Maps spatial coordinates (latitude and longitude). To prevent data anomalies and ensure strict CRUD compliance, it relies on Foreign Keys from the **USERS** table (identifying the reporter) and the **CATEGORIES** table (ensuring standardized item classifications).
- AI & Logic Tables (**AI_TAGS** and **MATCH_ALERTS**): To support the system's automated intelligence, the **AI_TAGS** table shares a one-to-many relationship with **ITEMS**, dynamically storing multiple descriptive labels and confidence levels generated by the OpenCV Vision API. The **MATCH_ALERTS** table logs the results of the back-end calculation engine, successfully bridging two separate records (a "Lost" item and a "Found" item) when their match score exceeds the 80% threshold.
- Security & Handover (**CLAIMS**): This table explicitly handles the secure resolution of an item. It records the entire OTP handover lifecycle by linking the specific **item_id**, the **claimant_user_id** (Student), and the **security_guard_id** (Admin), alongside a timestamp and the generated OTP code.
- System Monitoring (**API_LOGS**): Crucial for System Admin oversight, this table tracks the health of external webhooks (Google Maps and Telegram). By directly linking the **item_id** to the **API_LOGS** table, the back-end can trace exactly which user report triggered specific API requests, ensuring highly traceable system operations and error handling.

7. USER INTERFACE

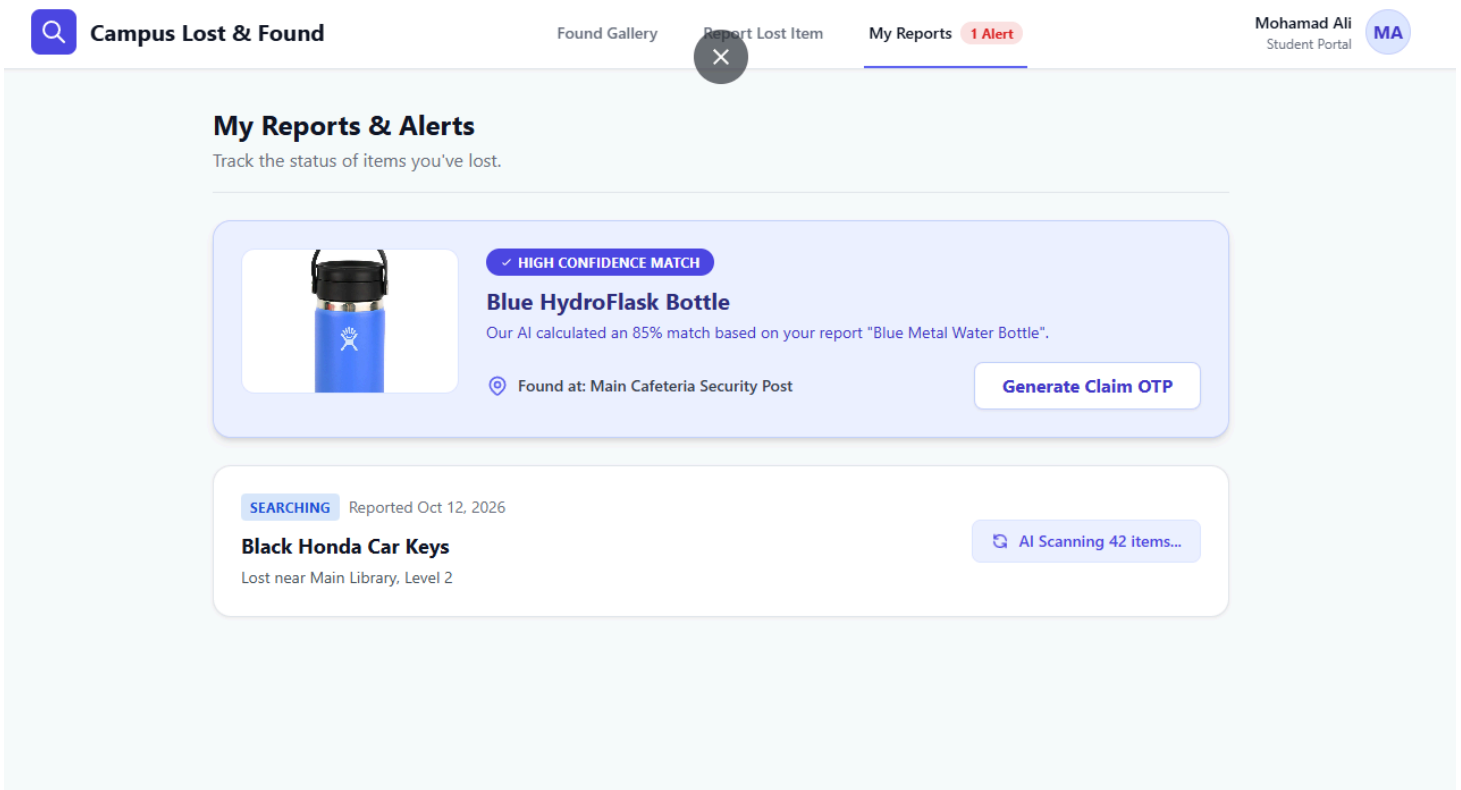


1. Found Gallery (Discovery Hub)

This is the primary landing page where students can browse items that have already been found and handed over to security.

- **AI-Enhanced Search:** The search bar allows users to query the database using natural language or specific tags generated by the OpenCV Vision API (e.g., "blue bottle" or "Honda keys").

- **Dynamic Categorization:** Navigation chips allow for instant filtering by item type (Electronics, Wallets, etc.), utilizing the **CATEGORIES** table logic from the database.
- **Visual Grid:** Each card displays the item's photo (retrieved from the VPS), its location based on Google Maps data, and a "time-ago" stamp to help students identify their belongings at a glance.



2. Report Lost Item (Submission Portal)

This interface handles the Create operation of the **ITEMS** CRUD module, allowing students to register a lost item into the system.

- **Detailed Parameters:** The form captures the item title, category, and unique distinctive features to feed into the matching algorithm.
- **Geospatial Input:** Instead of typing a location, users interact with a Google Maps API window to drop a pin on the "Last Known Location." This provides the exact latitude and longitude coordinates required for the proximity-based matching calculation.
- **Webhook Integration:** A toggle allows students to opt-in for Telegram Bot notifications, linking their **telegram_chat_id** to the report for real-time alerts.



Report a Lost Item

Provide details so our AI matching engine can actively search for it.

Item Title / Description

e.g., Black Honda Car Keys with Red Tag

Category

Electronics & Gadgets

Distinctive Features (Optional)

Any specific scratches, stickers, or marks that make it unique?



Get Notified via Telegram Bot

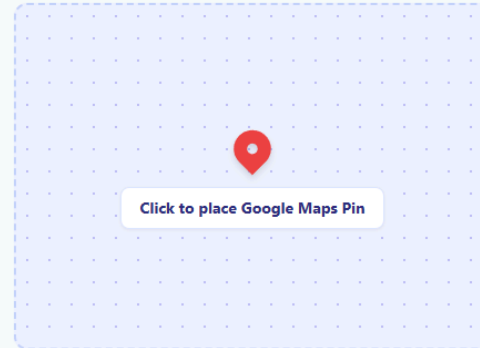
Our AI Matching Engine will instantly alert your linked Telegram account if we find a match.



Enable Webhooks

Last Known Location

Click on the map to drop a pin. This helps our algorithm calculate proximity to found items.



Start AI Search Process

3. My Reports & Alerts (Tracking & Resolution)

This view serves as the personal dashboard for students to monitor the status of their active reports.

- **Real-Time Status Tracking:** Reports are categorized by their current state, such as "Searching" (where the AI is actively scanning new entries).
- **High-Confidence Matches:** When the back-end Matching Calculation exceeds the 80% threshold, the system displays a "High Confidence Match" alert. It shows the student a comparison between their report and the found item.
- **Secure Handover (OTP):** If the student recognizes the item, they can click "Generate Claim OTP." This triggers the back-end to create a unique code in the **CLAIMS** table, which the student will present to Campus Security for a verified and secure handover.



Account Settings

Manage your profile and notification preferences.



Update Photo

Full Name

Mohamad Ali

Matric Number

2024123456

Locked

Telegram Handle

@ mohamad_ali99

Telegram Match Alerts

Receive automated notifications when our AI finds a high-confidence match for your items.

☒

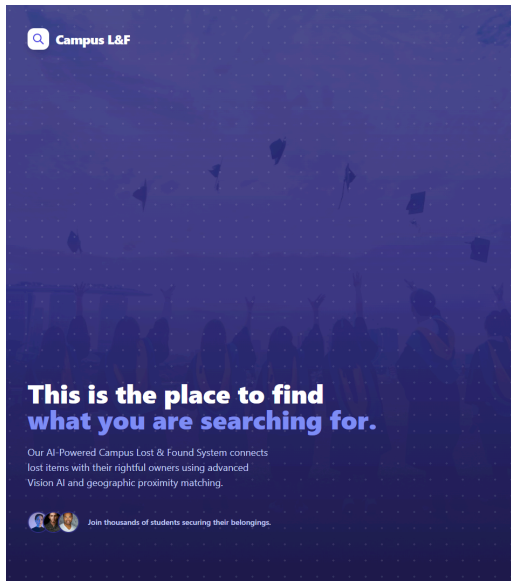
Save Changes

Sign Out

4. Account Settings (Profile & Preference Management)

This view serves as the personal configuration dashboard for students to manage their identity and notification settings.

- **Profile Management & Security:** The form allows users to update their full name and profile photo. Crucially, the "Matric Number" field is locked (read-only) to maintain strict database integrity and prevent identity spoofing within the campus database.
- **External Communication Setup:** Students can input and update their "Telegram Handle," providing the exact destination needed for the back-end to route automated messages.
- **Automated Match Alerts:** A dedicated toggle allows students to opt-in or opt-out of "Telegram Match Alerts." When active, it authorizes the system to send real-time push notifications to their device the moment the AI finds a high-confidence match for their items.
- **Secure Account Actions:** Distinct buttons at the bottom allow the user to commit their updates to the system via "Save Changes" or securely terminate their session by clicking "Sign Out."



Student Portal
Access your reports or create a new account.

[Sign In](#) [Create Account](#)

Matric Number

Password
 [Forgot password?](#)

☐ Remember me

[SIGN IN](#)

Student Portal
Access your reports or create a new account.

[Sign In](#) [Create Account](#)

Full Name

Matric Number Telegram Handle

Password

☐ I agree to the University [Terms of Service](#) and privacy guidelines.

[CREATE ACCOUNT](#)

5. Authentication & Onboarding Gateway (Student Portal)

These interfaces represent the secure entry point for the Student Portal. Designed with a modern, split-screen layout, the UI balances system branding and value proposition on the left with a frictionless, interactive form on the right.

Built as a Single Page Application (SPA), the interface handles user authentication and database registration seamlessly:

- **Dynamic State Toggling:** Utilizing Vue.js reactivity, the form features a seamless toggle switch between "Sign In" and "Create Account." This allows users to navigate between

authentication and registration states instantly without triggering a full page reload, ensuring a smooth user experience.

- **System Branding & Value Proposition:** The left panel serves as a visual anchor, reinforcing the system's core capabilities. It highlights the use of "advanced Vision AI and geographic proximity matching" to set clear expectations for the user before they even log in.
- **Secure Authentication (Sign In):** The login state offers a streamlined interface requiring the student's **Matric Number** and Password. Utilizing the Matric Number as the primary credential ensures that access is restricted strictly to verified university students, maintaining database security.
- **Strategic User Onboarding (Create Account):** The registration state maps directly to the backend **USERS** table schema. Crucially, it captures the **Telegram Handle** immediately during onboarding. By collecting this data upfront, the system ensures that new users are instantly "webhook-ready" to receive automated AI Match Alerts via the **@UiTMLostBot** the moment their account is provisioned.
- **Compliance & Validation:** The onboarding form includes a mandatory agreement to the university's Terms of Service and privacy guidelines, ensuring data compliance before any **INSERT** operation is executed in the database.

Security Portal

MAIN MENU

Inventory

Match Alerts 2

Api Logs

Settings

HQ Haziq Naeqib

Inventory

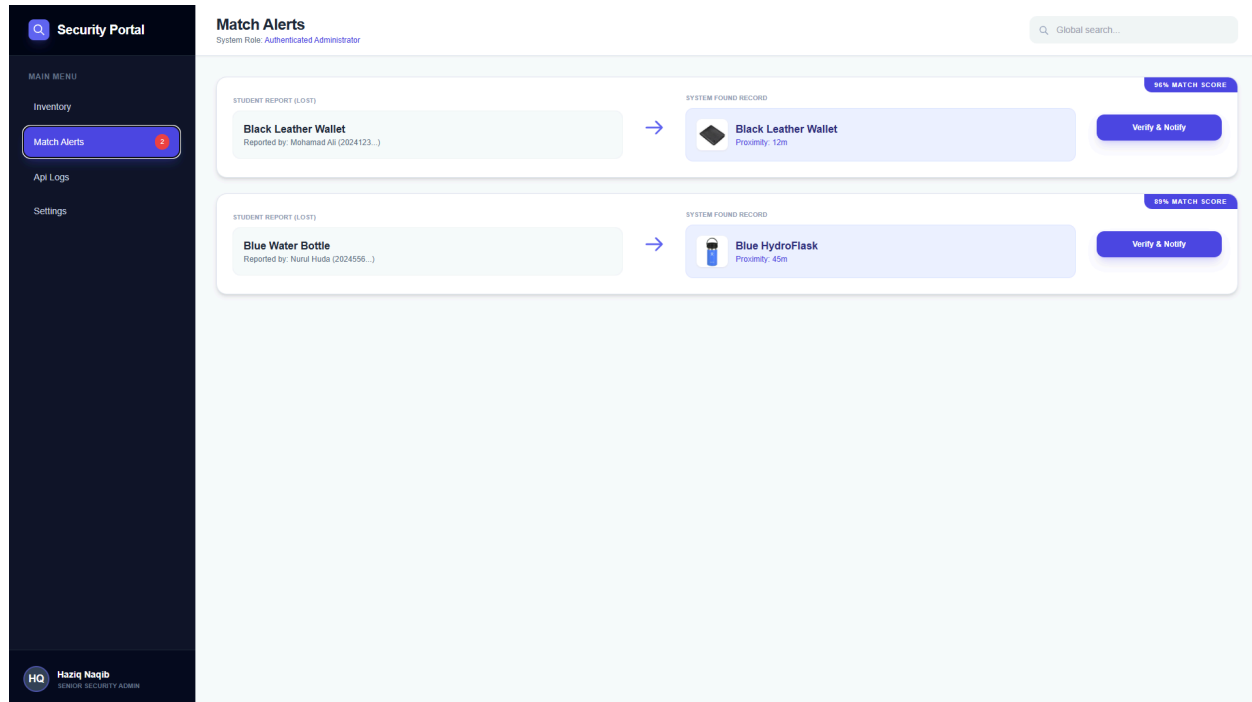
System Role: Authenticated Administrator

Global search...

ENTRY ID	VISUAL EVIDENCE	ITEM DESCRIPTION	PROXIMITY / LOCATION	STATUS	
#1001		Black Leather Wallet Wallets	Cafeteria A 3.8007, 345.5837	MATCHED	Audit Details
#1002		Blue HydroFask Accessories	Gymnasium 3.8722, 345.5812	PENDING	Audit Details
#1003		Honda Keys (Red Tag) Keys	Block B Carpark 3.8685, 345.5835	MATCHED	Audit Details
#1004		MacBook Pro Charger Electronics	Library Level 3 3.8788, 345.5859	CLAIMED	Audit Details
#1005		Matric Card (Ahmad) IDs	Main Gate 3.8756, 345.5889	PENDING	Audit Details
#1006		Nike Sports Bag Bags & Backpacks	Stadium 3.8768, 345.5838	PENDING	Audit Details
#1007		Reading Glasses Accessories	Library Level 1 3.8782, 345.5842	PENDING	Audit Details
#1008		Scientific Calculator Electronics	Lecture Hall 4 3.8823, 345.5838	CLAIMED	Audit Details

1. Inventory (Global Audit Hub) This is the primary workspace where security personnel can monitor and manage all items registered in the system's database.

- **Visual Evidence & Metadata:** The data table displays the item's photo alongside a strict Entry ID (e.g., #1001), item description, and categorical metadata mapped directly from the database.
- **Proximity / Location Tracking:** To verify spatial accuracy, the UI displays both the human-readable location and the precise latitude/longitude coordinates extracted via the Google Maps API.
- **Lifecycle Status & Auditing:** Dynamic badges (Pending, Matched, Claimed) track the item's current state. The "Audit Details" button allows staff to dive deeper into the raw system data, including the OpenCV tags.



2. Match Alerts (AI Decision Support) This interface bridges the gap between the automated AI calculations and the human verification process.

- **Side-by-Side Comparison:** Presents a direct visual and textual comparison between the "Student Report (Lost)" and the "System Found Record" to aid in fast identification.
- **Confidence Scoring:** Displays the exact Match Score (e.g., 96%) generated by the back-end matching engine, providing mathematical proof of the similarity between the two records.
- **Verification Workflow:** A prominent "Verify & Notify" button allows security personnel to manually confirm the AI's findings. Clicking this triggers the system to send the secure OTP to the student.

Security Portal

MAIN MENU

Inventory

Match Alerts

Api Logs

Settings

HQ

Haziq Naqib

SENIOR SECURITY ADMIN

Api Logs

System Role: Authenticated Administrator

Global search...

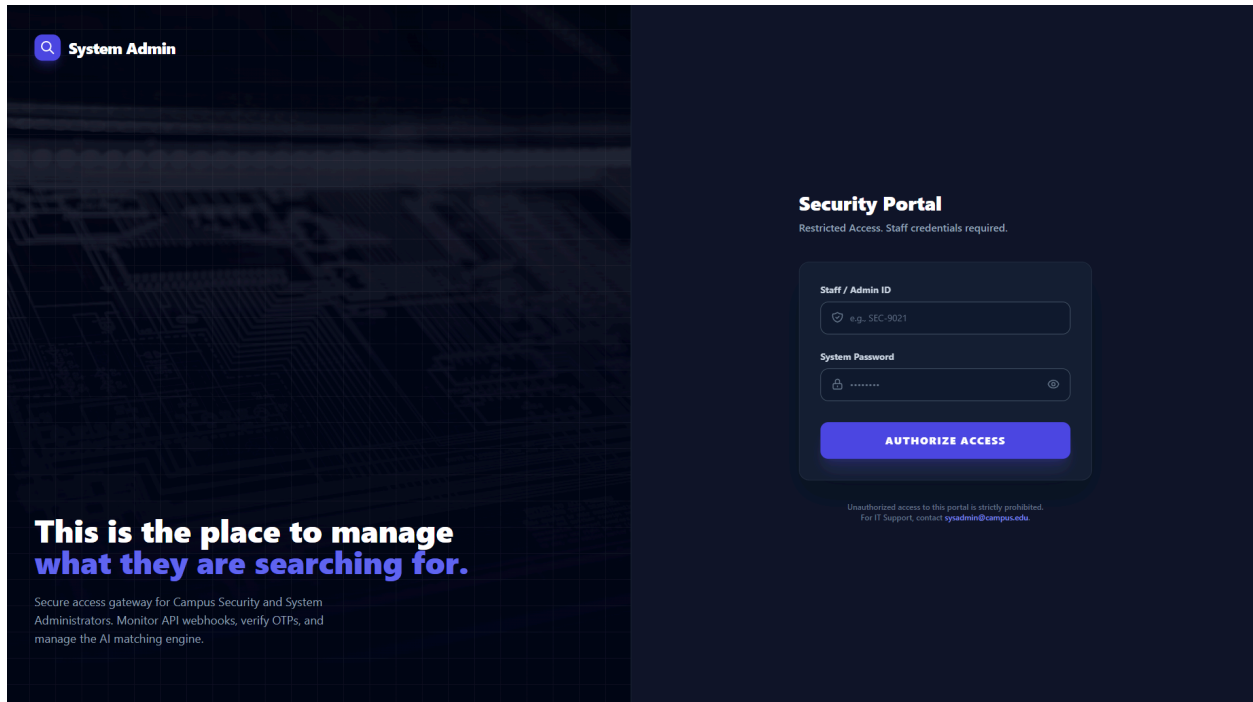
Real-time External Webhook Transactions

SYSTEM LIVE

SERVICE	ENDPOINT	STATUS	RESPONSE TIME	TIMESTAMP
VISION	POST /v1/images:annotate	200 OK	1420ms	2020-05-10 21:05:12
MAPS	GET /maps/api/geocode/json	200 OK	240ms	2020-05-10 21:05:14
BOT	POST /bot_TOKEN/sendMessage	200 OK	110ms	2020-05-10 21:06:01
VISION	POST /v1/images:annotate	200 OK	1560ms	2020-05-10 21:06:44
MAPS	GET /maps/api/geocode/json	200 OK	310ms	2020-05-10 21:06:45

3. API Logs (Technical Monitoring) This view serves as the technical oversight dashboard, specifically designed for the System Admin role to monitor infrastructure health.

- **Real-Time Transaction Tracking:** Logs every outgoing request to external webhooks, explicitly tracking interactions with the Vision API, Maps API, and Telegram Bot.
- **Performance Metrics:** Displays HTTP status codes (e.g., 200 OK) and response times (e.g., 1420ms) to help the admin track system latency and ensure service uptime.
- **Timestamped Traceability:** Every API call is logged with an exact timestamp, ensuring a complete, searchable audit trail of how the system’s backend modules are performing.



6. Security & Administrative Access Gateway (Admin Login)

This interface serves as the highly restricted entry point specifically designed for Campus Security and System Administrators. To instantly differentiate it from the student-facing portal, the UI employs a dark-themed, high-contrast aesthetic that conveys authority, backend management, and strict security.

Built within the same SPA architecture, this view handles elevated authentication processes:

- **Role-Based Access Control (RBAC):** Unlike the student portal, this interface deliberately omits a "Create Account" option. Because administrative roles are strictly provisioned by the university's IT department, the UI enforces a closed-loop authentication system requiring a pre-registered **Staff / Admin ID**. This directly maps to the `enum | role (Admin/Security)` constraint in the `USERS` database table.
- **Contextual Branding & Scope:** The left-hand panel reinforces the operational responsibilities of the staff. By explicitly stating that this portal is used to "Monitor API webhooks, verify OTPs, and manage the AI matching engine," it immediately sets the technical context for the user.
- **Focused Authentication UI:** The right-hand panel provides a distraction-free, secure login form. The use of specialized terminology, such as "System Password" and the

"Authorize Access" call-to-action button, emphasizes the elevated security privileges associated with entering the backend dashboard.

- **Unauthorized Access Deterrence:** The inclusion of a strict unauthorized access warning and an IT support contact link at the bottom adds an essential layer of compliance and administrative protocol required for enterprise-level university systems.